

LAB GUIDELINES

Lab 28 — Troubleshooting Methodology Walk-Through

Maps to CompTIA Server+ objective 4.1 — Explain the troubleshooting theory and methodology:

1. Identify the problem and determine the scope.
 - Question users / stakeholders, identify changes
 - Collect documentation / logs
 - Replicate the problem
 - Perform backups before making changes
 - Escalate if necessary
2. Establish a theory of probable cause (question the obvious).
 - Determine if there is a common element or symptom causing multiple problems
3. Test the theory to determine the cause.
4. Establish a plan of action to resolve the problem (notify impacted users).
5. Implement the solution or escalate.
 - One change at a time, test/confirm
 - If not resolved, reverse the change and try a new one
6. Verify full system functionality and implement preventive measures.
7. Perform a root cause analysis.
8. Document findings, actions, and outcomes throughout the process.

Run all commands on <https://killercoda.com/playgrounds/scenario/ubuntu>

Step 1 — Manufacture a fault to troubleshoot

Use a real service so the walk-through has teeth. Install nginx and then break it:

```
apt update && apt install -y nginx
systemctl start nginx
curl -sI http://127.0.0.1/ | head -1      # should be 200/301

# Break it: change listen port to one that's already in use
sed -i 's/listen 80;/listen 22;/' /etc/nginx/sites-enabled/default
systemctl restart nginx 2>&1 | head -5
systemctl is-active nginx
```

The user-reported symptom: **"The website is down."**

Step 2 — Identify the problem (scope)

Questions a Server+ technician should ask the user:

- When did it start? (relative + absolute time)
- Was anything changed recently? (link to Lab 17 change log)
- Is *anyone else* affected? (single-user, dept, all?)
- What error or page do you see?
- Can you reproduce it?

Collect on your side:

```
journalctl -u nginx -n 40 --no-pager
tail -30 /var/log/syslog
systemctl status nginx
```

Scope: nginx itself is failing — single service, every user.

Step 3 — Replicate

```
curl -sI http://127.0.0.1/ 2>&1 | head
```

```
nc -vz 127.0.0.1 80
```

Both fail. Reproducible from the server itself → not a network or client issue.

Step 4 — Back up before changes

```
cp /etc/nginx/sites-enabled/default /root/default.pre-fix.$(date +%s)
ls /root/default.pre-fix.*
```

Server+ 4.1 explicitly: *perform backups before making changes*.

Step 5 — Establish a theory (question the obvious)

The journalctl output likely shows:

```
nginx: [emerg] bind() to 0.0.0.0:22 failed (98: Address already in use)
```

The "obvious" theory: **port 22 is occupied by sshd**. Confirm with one command:

```
ss -tlnp 'sport = :22'
```

That single line proves the theory.

Step 6 — Plan of action and user notification

Plan:

1. Revert nginx listen port to 80.
2. Reload nginx.
3. Verify HTTP 200.
4. Document and close ticket.

Notify users: "Website restoration in progress, ETA 5 min."

Step 7 — Implement the solution (one change at a time)

```
sed -i 's/listen 22;/listen 80;/' /etc/nginx/sites-enabled/default
nginx -t
systemctl restart nginx
```

If this had not worked, the rule from 4.1 is to **reverse the change** and try a new theory.

Step 8 — Verify full system functionality

```
curl -sI http://127.0.0.1/ | head -1          # 200 OK
ss -tlnp 'sport = :80'                       # nginx listening
systemctl is-active nginx                    # active
journalctl -u nginx --since "5 minutes ago" | tail -5  # no new errors
```

"Full system" includes the **dependencies**: the firewall (Lab 9) still allows 80, the DNS record still resolves, the load balancer (Lab 14) still routes.

Step 9 — Preventive measure

Add a config-validation step to deployment so the same fault doesn't reappear:

```
cat > /etc/cron.d/nginx-config-check <<'EOF'
*/5 * * * * root nginx -t >/var/log/nginx-config-check.log 2>&1
EOF
```

Better preventive: gate every nginx config change behind a CI check that runs `nginx -t` against the proposed file.

Step 10 — Root cause analysis

The proximate cause: someone edited `listen` to 22. The **root cause** is usually further up the chain:

5-whys	Answer
Why did nginx fail?	Listen port conflict with sshd

5-whys	Answer
Why did the listen port get changed?	A copy-paste from a sandbox config
Why did the bad config reach production?	No CI check on config edits
Why was there no CI check?	No pipeline ownership defined
Why was no pipeline ownership defined?	New role wasn't filled after re-org

Action items: **fill the pipeline-owner role, add nginx -t to CI.**

Step 11 — Documentation

```
Ticket:    INC-2026-0521-001
Summary:   nginx down 10 min – listen port conflict
Cause:     Bad copy-paste of dev config to prod (listen 22)
Fix:       Reverted listen to 80, reloaded service
Time to detect: 3 min (monitoring alert)
Time to resolve: 7 min
Preventive: nginx -t in CI pipeline (owner: web team)
Attached:  pre-fix config, post-fix config, journalctl excerpt
```

File this in the same place as the change log from Lab 17 — every incident must leave a paper trail.

Free online tools

- ****CompTIA troubleshooting methodology (free PDF excerpt)**** — <https://www.comptia.org/>
- ****ITIL Incident Management overview**** — <https://www.atlassian.com/itsm/incident-management>
- ****SANS root-cause analysis cheat sheet**** — <https://www.sans.org/>

What you learned

- The Server+ 7-step troubleshooting model applied end-to-end to a real fault.
- Where each step's evidence comes from on Linux (`journalctl`, `ss`, `systemctl`, `curl`).
- The discipline of one-change-at-a-time and the reverse-if-it-didn't-help rule.
- A reusable RCA / 5-whys template and a documented post-incident record.